Addition 4b: Verifying the Parallelism Mechanism

Local PostgreSQL 15.17, genuine TPC-H SF1 data (6,001,215 lineitem rows, exact match to the collection guide's documented count), generated via DuckDB's `dbgen` and loaded/indexed/analyzed in Postgres. All 21 patched queries run with `EXPLAIN (ANALYZE, VERBOSE, FORMAT JSON)`.

Config check (done first, per the task's own instruction)

setting	value
`max_parallel_workers_per_gather`	**2**
`max_parallel_workers`	8
`max_worker_processes`	8

This is PostgreSQL's factory default (unmodified since PG10), left untouched on this local install. Outcome (c) — parallelism disabled during collection — is **ruled out** to the extent it can be: the original 5 EC2 collection instances are already terminated and cannot be re-queried directly, so this is an *inference*, not a re-measurement — but the CETP collection guide (read in full in an earlier session) documents every setup step exhaustively and never once mentions tuning parallel-query settings, and there is no evidence anywhere in the project files of a custom `postgresql.conf`. Absent evidence of deliberate tuning, the collection instances almost certainly ran at this same default of 2. This is stated as an inference with a disclosed gap, not a confirmed fact.

Per-query parallel-plan presence

19 of 21 queries use parallel execution (`Gather` or `Gather Merge` nodes present, 2 workers launched each — matching `max_parallel_workers_per_gather=2`). Only **q11 and q20 run fully serially** (no parallel nodes at all). Full per-query table: `results/addition4b_parallel_plan_check.csv`.

The core test: does parallel-plan status predict which queries benefited from the multi-core signal?

Per-query benefit = (MAPE using OLD single-core compute_ratio in the analytical formula) − (MAPE using NEW multi-core compute_ratio), computed on each query's 20 dev→prod pairs. Positive = multi-core signal helped.

query	benefit (pp MAPE reduction)	uses parallel plan
q1	+47.4	yes
q12	+40.2	yes
q16	+32.6	yes
q5	+24.9	yes
q8	+23.7	yes
q9	+22.1	yes
q3	+22.0	yes
q10	+18.9	yes
q18	+18.4	yes
q21	+5.9	yes
q6, q4, q20, q22, q14	≈0	mixed (q20 is the serial one)
q13, q19, q17	≈0 to −0.2	yes
q7	−8.4	yes
q11	**−22.0**	**no (serial)**
q2	**−25.1**	yes

Group means: **parallel-plan queries +11.70pp mean benefit; serial-plan queries −11.02pp mean benefit** (n=2: q20 at exactly 0.0, q11 at −22.0). Direction matches the hypothesis, but this is barely a sample (n=2), and it's essentially one data point (q11) doing all the work — q20 shows no benefit and no harm.

This is not a clean confirmation. Two things complicate it:

1. **The strongest driver of "benefit" is bottleneck class, not parallel-plan status.** The analytical formula only gives `compute_ratio` real weight when a query's Phase-5 bottleneck probability assigns it substantial `p_compute` or `p_mixed` — structurally, `bandwidth` / `io`-labeled queries get ~0 weight on compute regardless of whether their plan happens to be parallel (q6, q4, q14, q13, q19, q17, q22 all show ≈0 benefit despite most of them running parallel plans). The clean top-of-ranking pattern — q1, q12, q16 (exactly the three Phase-1 `compute`-labeled queries) showing the largest benefit by a wide margin — is as much a re-confirmation of the SQL-structure-based bottleneck labeling from Task 1 as it is independent evidence for the parallelism mechanism specifically.
2. **Two parallel-plan queries (q2, q7) show negative benefit**, which the parallelism hypothesis does not predict — if parallel execution were the clean explanation, a parallel-plan query with real compute weight should benefit, not get worse. With only 21 queries and 20 pairs each, this is plausibly small-sample noise rather than a real counter-mechanism, but it cannot be ruled out with this data, and it should not be explained away without saying so.

Verdict on the core test: leaning-confirmed but not clean. The config check rules out outcome (c) (parallelism-was-off) directly. The gross direction (parallel-plan queries benefit more on average) matches the hypothesis, and the one genuinely serial-and-compute-weighted query (q11) shows a large negative benefit, consistent with "no parallel workers → multi-core throughput shouldn't matter, and it doesn't help." But the evidence is not strong enough to call this a clean, fully-established mechanism: the serial-plan sample is too small (n=2, and one of those two is a null result, not a positive counter-example), and two parallel-plan queries (q2, q7) go the wrong direction without an identified explanation. **This is outcome (a)-leaning but reported with its real limits, not forced into a clean confirmation.**

Confound check: measured throughput vs. raw vCPU count

	correlation with scaling_factor_p50	LOMO analytical MAPE	beats naive-linear (23.03%)?
measured multi-core GFLOPS	**−0.691**	**21.60%**	yes
raw vCPU count ratio (2 or 4, no measurement)	−0.479	26.32%	**no**

Measured throughput clearly beats raw vCPU count as a predictor. A naive "just count the cores" feature is meaningfully weaker (correlation magnitude 0.479 vs 0.691) and — critically — using vCPU count alone in the analytical formula does **not** beat naive-linear (26.32% vs 23.03%), while the measured benchmark does (21.60%). **This confound is ruled out**: the compute benchmark is capturing real, measured throughput differences (e.g., per-core clock/microarchitecture differences between c5a's and m5a's shared 4-vCPU count, or c7i's/r5n's/z1d's shared 2-vCPU count) beyond what a trivially-available integer feature would give you. The instrument adds value; it isn't just a fancy proxy for core count.

Honest overall verdict

- **Parallelism was almost certainly active during collection** (config inference, not directly re-verified against the terminated instances) — outcome (c) is ruled out.
- **The parallelism mechanism is partially, not cleanly, confirmed.** The one truly diagnostic case (q11: serial plan, compute-weighted, and it's the second-worst-hurt query by the multi-core signal) is consistent with the hypothesis. But the evidence set as a whole is thin (only 2 serial-plan queries exist in this 21-query workload) and includes two unexplained counter-examples (q2, q7) among the parallel-plan queries. **Report this as leaning-confirmed-but-incomplete, not as an established mechanism** — a stronger test would need a workload with a better balance of serial vs. parallel query plans, which TPC-H at SF1 under default settings does not provide (19 of 21 queries parallelize).
- **The confound check is clean and unambiguous**: measured multi-core throughput is a better predictor than raw vCPU count, so whatever is driving the Addition 4 result, it is not merely "bigger instances have more cores" — the benchmark is measuring something real.

- **What this changes about Addition 4's claim**: the mechanism sentence should be softened from "plausibly because of PostgreSQL parallel query execution" to "consistent with, but not conclusively established as, a parallel-query-execution effect — the config supports it being active, the one clear serial-query case supports it, but the small serial-query sample and two unexplained exceptions mean this remains a well-evidenced hypothesis rather than a confirmed mechanism." The core Addition 4 finding itself (multi-core compute signal statistically beats naive-linear) is unaffected by this — it stands on its own regression evidence regardless of which exact mechanism explains it.

Artifacts

- `scripts/collection/tpch_data/` — local SF1 TPC-H data (DuckDB-generated) and Postgres load script.
- `scripts/collection/check_parallelism.py` — EXPLAIN-based parallel-node detector for all 21 queries.
- `addition4b_mechanism_check.py` — per-query benefit ranking, mechanism test, confound check.
- `results/addition4b_parallel_plan_check.csv` — per-query parallel-node presence and worker counts.
- `results/addition4b_per_query_benefit.csv` — per-query benefit ranking merged with parallel-plan status.
- `results/addition4b_mechanism_summary.csv` — summary statistics.